MATERIAL COMPLEMENTARIO DE ESTUDIO

Despliegue de Next.js con PostgreSQL

Render con contenedor Docker y alternativa con Vercel + Neon

Diseño y Desarrollo Web · Prácticas Profesionalizantes II

Guía práctica paso a paso · Planes gratuitos · Actualizado al 3 de noviembre de 2026

1. Cómo utilizar este material

Este material complementa la aplicación TODO construida con Next.js y PostgreSQL. El objetivo es publicarla en Internet, comprender los servicios involucrados y configurar la conexión entre la aplicación y la base de datos.

Objetivos de aprendizaje

• Distinguir aplicación web, contenedor, hosting y base de datos administrada.

• Desplegar PostgreSQL y Next.js como servicios separados en Render.

• Construir una imagen Docker y configurar el puerto de escucha.

• Configurar variables de entorno sin exponer credenciales.

• Desplegar Next.js en Vercel y conectarlo con PostgreSQL en Neon.

• Comparar las alternativas gratuitas y sus limitaciones.

Recorrido

Etapa	Resultado
Preparación	Proyecto Next.js funcional y repositorio GitHub.
Render PostgreSQL	Base creada y tabla tasks disponible.
Docker + Render	Servicio web desplegado desde una imagen.
Conexión	Next.js consulta y modifica PostgreSQL remoto.
Vercel + Neon	Alternativa administrada desplegada.
Validación	URL pública y persistencia comprobadas.

Alcance: se asume Next.js App Router, JavaScript y acceso a PostgreSQL mediante `pg` y la variable `DATABASE_URL`.

2. Conceptos antes de desplegar

¿Qué significa desplegar?

Desplegar consiste en publicar una aplicación en infraestructura accesible por Internet. En este ejercicio existen dos componentes: Next.js procesa las solicitudes HTTP y PostgreSQL almacena los datos de forma persistente.

¿Qué aporta Docker?

Docker empaqueta la aplicación y sus dependencias en una imagen. Render ejecuta esa imagen como servicio web. El filesystem del contenedor gratuito es efímero: no se deben guardar allí datos que deban conservarse.

Variables de entorno

Una variable de entorno permite configurar la aplicación sin escribir credenciales en el código. La cadena de conexión se guardará como `DATABASE_URL` en el panel del proveedor. En local puede utilizarse `.env.local`, que no debe subirse a Git.

Aspecto	Render	Vercel
Ejecución	Servicio web Docker.	Plataforma administrada para Next.js; no ejecuta nuestro Dockerfile.
Base de datos	Render PostgreSQL.	Proveedor externo; en esta guía, Neon PostgreSQL.
Conexión	URL interna entre servicios en misma región.	DATABASE_URL desde integración o variable de entorno.

Aspecto	Render	Vercel
Aprendizaje	Contenedores, servicios y redes.	Despliegue Git y funciones administradas.

3. Preparar el proyecto

• Comprobar que `npm run dev` inicia la aplicación y que el CRUD funciona localmente.

• Verificar que `.env.local` esté en `.gitignore`.

• Subir al repositorio el código, `package.json` y el lockfile correspondiente.

• Utilizar `process.env.DATABASE_URL`; no escribir credenciales directamente en el código.

Módulo de conexión

```
// lib/db.js
import "server-only";
import pg from "pg";

const { Pool } = pg;
if (!process.env.DATABASE_URL) {
  throw new Error("Falta configurar DATABASE_URL");
}
const pool = new Pool({
  connectionString: process.env.DATABASE_URL
});
export default pool;
```

La configuración SSL depende del proveedor. Render y Neon documentan sus requisitos; seguir la configuración vigente de cada uno. La conexión y las credenciales deben permanecer en código de servidor, nunca en un Client Component.

4. Alternativa A: desplegar en Render

Se crearán dos recursos: una base PostgreSQL administrada y un servicio web que ejecuta el contenedor Docker de Next.js. Conviene elegir la misma región para ambos.

4.1. Crear PostgreSQL

• Ingresar a Render Dashboard.

• Seleccionar New → PostgreSQL.

• Asignar un nombre, por ejemplo `todo-postgres`.

• Elegir la región y el plan Free, si está disponible.

• Crear la base y esperar a que esté disponible.

4.2. Obtener la URL de conexión

Abrir la base y seleccionar Connect. Render muestra una URL interna y otra externa. La URL interna se utiliza desde servicios Render de la misma cuenta y región. La externa sirve para conectar herramientas que se ejecutan fuera de Render.

```
postgresql://usuario:contraseña@host-interno:5432/nombre_db
```

Copiar la Internal Database URL para el servicio web. Tratarla como una contraseña: no pegarla en el código, capturas públicas ni repositorios.

4.3. Crear la tabla

Conectarse con un cliente PostgreSQL utilizando la URL externa, o el SQL Editor disponible en el proveedor, y ejecutar el SQL de la tabla que utiliza la aplicación:

```
CREATE TABLE IF NOT EXISTS tasks (
  id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  title VARCHAR(200) NOT NULL,
```

```
  completed BOOLEAN NOT NULL DEFAULT FALSE,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  CONSTRAINT tasks_title_not_blank
    CHECK (LENGTH(TRIM(title)) > 0)
);
```

4.4. Configurar Next.js para Docker

En `next.config.js`, activar la salida standalone:

```
/** @type {import('next').NextConfig} */
const nextConfig = { output: "standalone" };
module.exports = nextConfig;
```

Crear un `Dockerfile` en la raíz. Este ejemplo supone npm y `package-lock.json`:

```
FROM node:22-alpine AS deps
WORKDIR /app
COPY package.json package-lock.json ./
RUN npm ci

FROM node:22-alpine AS builder
WORKDIR /app
COPY --from=deps /app/node_modules ./node_modules
COPY . .
ENV NEXT_TELEMETRY_DISABLED=1
RUN npm run build

FROM node:22-alpine AS runner
WORKDIR /app
ENV NODE_ENV=production
ENV HOSTNAME="0.0.0.0"
COPY --from=builder /app/public ./public
COPY --from=builder /app/.next/standalone ./
COPY --from=builder /app/.next/static ./.next/static
EXPOSE 3000
CMD ["node", "server.js"]
```

Si el proyecto no tiene carpeta `public`, crearla vacía o quitar esa instrucción COPY. Para pnpm o yarn, adaptar la etapa de dependencias y el lockfile.

4.5. Crear .dockerignore

```
.dockerignore
node_modules
.next
.git
.env
.env.*
!.env.example
npm-debug.log
```

Así se evita copiar dependencias locales y secretos al contexto de construcción. No incluir una DATABASE_URL real dentro de la imagen.

4.6. Crear el Web Service

• Subir Dockerfile, next.config.js y .dockerignore a GitHub.

• En Render, seleccionar New → Web Service y conectar GitHub.

• Elegir el repositorio de la aplicación.

• Seleccionar la misma región que PostgreSQL.

• Seleccionar runtime Docker si el asistente lo solicita.

• Elegir el plan Free y crear el servicio.

Render construirá la imagen y ejecutará el comando de inicio. Next.js debe escuchar en `0.0.0.0`. El servidor standalone utiliza el puerto 3000 por defecto; si se configura otro puerto, debe coincidir con la configuración del servicio.

4.7. Configurar DATABASE_URL

• Abrir el Web Service en Render y entrar en Environment.

• Agregar una variable llamada DATABASE_URL.

• Pegar como valor la Internal Database URL de PostgreSQL.

• Guardar y realizar un nuevo deploy si no se inicia automáticamente.

La aplicación se conecta por la red interna de Render. No es necesario exponer PostgreSQL a Internet para esta comunicación. La conexión debe realizarse desde código de servidor.

4.8. Validar

• Esperar a que el deploy finalice y abrir la URL pública.

• Verificar que se listen tareas guardadas en PostgreSQL.

• Crear, editar, completar y eliminar tareas.

• Recargar y comprobar que los cambios persisten.

• Si falla, revisar los logs y verificar DATABASE_URL.

5. Límites del plan gratuito de Render

Recurso	Limitación relevante
Web Service	Se suspende tras 15 minutos sin tráfico; al llegar una solicitud puede tardar cerca de un minuto en reactivarse.
Horas	750 horas gratuitas por workspace y mes, compartidas por servicios Free.
PostgreSQL	1 GB de almacenamiento y una única base Free activa por workspace.
Vencimiento	PostgreSQL Free expira a los 30 días. Hay 14 días de gracia para actualizar antes de la eliminación.
Filesystem	Efímero; no conservar datos locales del contenedor.
Backups	Las bases Free no incluyen backups administrados.

Conclusión: Render es útil para aprender Docker, despliegue y comunicación entre servicios. No utilizar PostgreSQL Free para datos que deban conservarse más de 30 días. Exportar la información antes del vencimiento.

6. Alternativa B: Vercel + Neon

Vercel ofrece despliegue administrado para Next.js. En lugar de ejecutar el contenedor Docker creado para Render, detecta el framework, instala dependencias, compila y publica sus rutas y funciones. Para PostgreSQL se utilizará Neon.

6.1. Crear la base en Neon

• Ingresar a Neon y crear una cuenta.

• Crear un proyecto PostgreSQL llamado, por ejemplo, `todo-vercel`.

• Elegir una región cercana a la región de despliegue de Vercel.

• Abrir Connect y copiar la cadena de conexión recomendada para aplicaciones serverless.

```
postgresql://usuario:contraseña@host/neondb?sslmode=require
```

El ejemplo es ilustrativo: utilizar la URL real provista por Neon y conservar los parámetros SSL y de pooling que incluya.

6.2. Crear la tabla

Abrir el SQL Editor de Neon o conectarse con un cliente PostgreSQL y ejecutar el mismo SQL de creación de `tasks` utilizado en Render. La estructura de datos no cambia.

6.3. Importar el proyecto en Vercel

• Ingresar a Vercel.

• Seleccionar Add New → Project.

• Importar el repositorio GitHub de la aplicación.

• Verificar que Framework Preset detecte Next.js.

• Mantener los comandos de build e instalación detectados automáticamente.

• Seleccionar Deploy.

6.4. Conectar Vercel con Neon

Se puede utilizar la integración de Neon del Marketplace o configurar la variable manualmente.

• Integración: desde el proyecto Vercel, abrir Storage/Marketplace, buscar Neon, instalar la integración y crear o vincular una base.

• Manual: abrir Settings → Environment Variables, agregar DATABASE_URL y pegar la cadena de conexión de Neon.

Si se utiliza la integración, comprobar el nombre exacto de la variable generada. El código espera DATABASE_URL, por lo que esa variable debe existir en el entorno correspondiente.

6.5. Entornos y redeploy

Configurar al menos Production. Para pruebas, también pueden habilitarse Preview y Development. Después de guardar o cambiar una variable de entorno, generar un nuevo deployment para que la aplicación la reciba.

6.6. Validar

• Abrir la URL pública de Vercel.

• Probar las operaciones CRUD.

• Recargar y verificar que los datos persistan en Neon.

• Ante errores, revisar Build Logs y Runtime Logs y validar la cadena de conexión.

¿Qué ocurre con Docker?

El Dockerfile preparado para Render no es el mecanismo de despliegue en esta alternativa. Vercel utiliza su plataforma de build y ejecución para Next.js. Se reutilizan el código y DATABASE_URL, pero cambia la infraestructura.

7. Límites y comparación

Criterio	Render Free	Vercel Hobby + Neon Free
Costo inicial	USD 0 dentro de límites.	USD 0 dentro de cuotas.
Hosting	Servicio web Docker que se suspende por inactividad.	Hosting administrado para Next.js.
PostgreSQL	Render Free: 1 GB; vence a 30 días.	Neon Free: hasta 1 GB por proyecto según plan vigente.
Docker	Sí, se construye y ejecuta la imagen.	No se utiliza el Dockerfile.
Persistencia	La base tiene vencimiento.	Sujeta a cuotas y políticas de Neon.
Enfoque	Contenedores y comunicación entre servicios.	Despliegue Git y funciones administradas.

Vercel Hobby está orientado a proyectos personales y no comerciales. Verificar las condiciones antes de usarlo para un sitio comercial o empresarial. Los límites pueden cambiar; consultar los planes oficiales antes de comenzar.

¿Cuál conviene?

Para aprender contenedores y conexión entre servicios, elegir Render. Para publicar Next.js con menos configuración de infraestructura y una base PostgreSQL gratuita para una demostración, elegir Vercel + Neon, respetando las cuotas de ambos planes.

8. Problemas frecuentes

Síntoma	Qué revisar
La aplicación no inicia	Logs de build/runtime, versión de Node, Dockerfile y salida standalone.
DATABASE_URL no existe	Nombre exacto y entorno configurado; volver a desplegar.
Error PostgreSQL	Host, usuario, contraseña, nombre de DB, SSL, permisos y región.
Render tarda en responder	El servicio Free pudo suspenderse por inactividad.
No aparecen datos	Tabla creada en la base correcta; consulta y logs del servidor.
Los cambios desaparecen	Confirmar que se escribe en PostgreSQL y no en archivos o memoria.
Funciona local pero no en producción	Variables, SSL, configuración de build y código exclusivo de servidor.

Regla de seguridad: nunca colocar DATABASE_URL en una variable NEXT_PUBLIC_, nunca imprimir contraseñas en logs y nunca importar el módulo PostgreSQL desde un Client Component.

9. Actividad integradora

Consigna: desplegar la aplicación TODO del material anterior mediante una de las alternativas. Se debe demostrar que está disponible mediante una URL pública y que las operaciones CRUD persisten en PostgreSQL remoto.

Entregables

• URL pública de la aplicación.

• Proveedor y plan gratuito utilizado para web y base de datos.

• Captura del panel que muestre los servicios, ocultando credenciales.

• Evidencia de alta, consulta, modificación y eliminación.

• Explicación de cómo se configuró y protegió DATABASE_URL.

• Descripción de al menos dos límites del plan gratuito elegido.

Criterios de validación

• La aplicación abre desde un navegador externo.

• Los datos se consultan desde PostgreSQL remoto.

• Crear, actualizar y eliminar funciona.

• Los cambios persisten después de recargar.

• No se exponen credenciales en el repositorio.

• El equipo explica qué ejecuta el contenedor o por qué Vercel no lo utiliza.

10. Referencias oficiales

• Render — Deploy for Free — https://render.com/docs/free

• Render — Create and Connect to Render Postgres — https://render.com/docs/postgresql-creating-connecting

• Render — Docker — https://render.com/docs/docker

• Render — Pricing — https://render.com/pricing

• Vercel — Hobby Plan — https://vercel.com/docs/plans/hobby

• Vercel — Next.js — https://vercel.com/docs/frameworks/nextjs

• Neon — Pricing — https://neon.com/pricing

• Neon — Vercel integration — https://neon.com/docs/guides/vercel

Fecha de consulta: 3 de noviembre de 2026.